

# Announcements

- Exam 2
  - Friday, April 24
  - Covers Lectures 15-31
  - Multiple-choice (closed-notes): terminology and basic concepts
  - Short-answer questions (open-notes): homework exercises, assignments
- Cluster Presentations
  - Wednesday, April 29
  - Lightning talk (7 min / team)
- Term Project
  - To be posted on Wednesday, April 22
  - Due May 7

CPSC 375 High-Performance Computing

Lecture 3I

# Matrix Multiplication II

Parallel Algorithms

# Matrix Multiplication

Let  $A$  and  $B$  be  $n$ -by- $n$  matrices:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \quad B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{pmatrix}$$

$C = AB$  is defined by:

$$C = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{pmatrix}$$

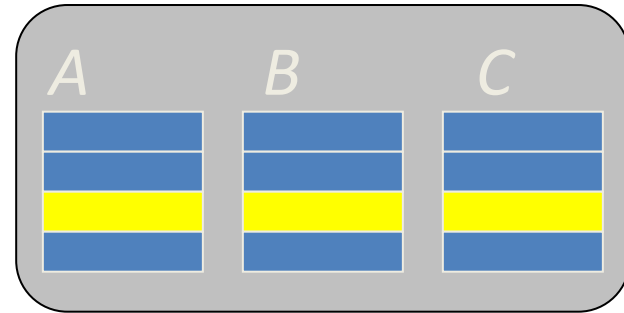
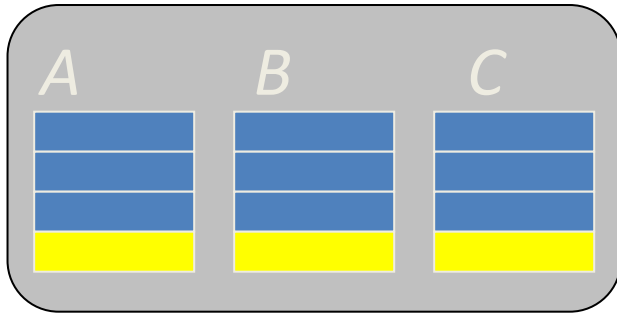
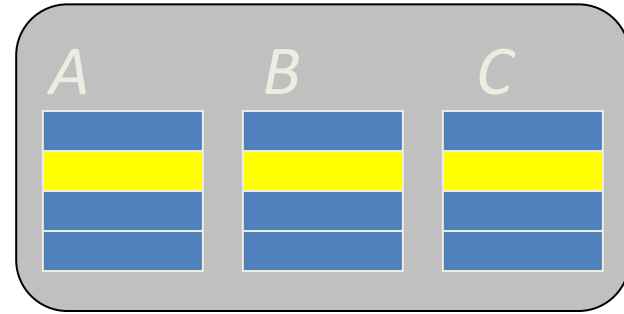
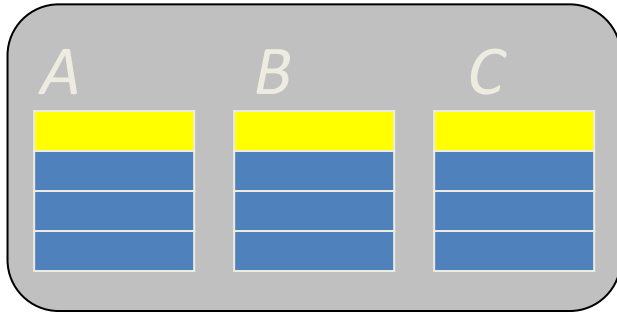
where

$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}, \quad 1 \leq i, j \leq n \Rightarrow O(n^3)$$

# Block-Striped Matrix Multiplication

- Partitioning
  - Divide matrices into  $p$  horizontal stripes of size  $(n/p) \times n$ .
  - Process  $i$  holds the  $i$ -th stripe of  $A$ ,  $B$ , and  $C$ , denoted as  $A_i$ ,  $B_i$ , and  $C_i$ , resp.
  - Each process has corresponding rows of three matrices.

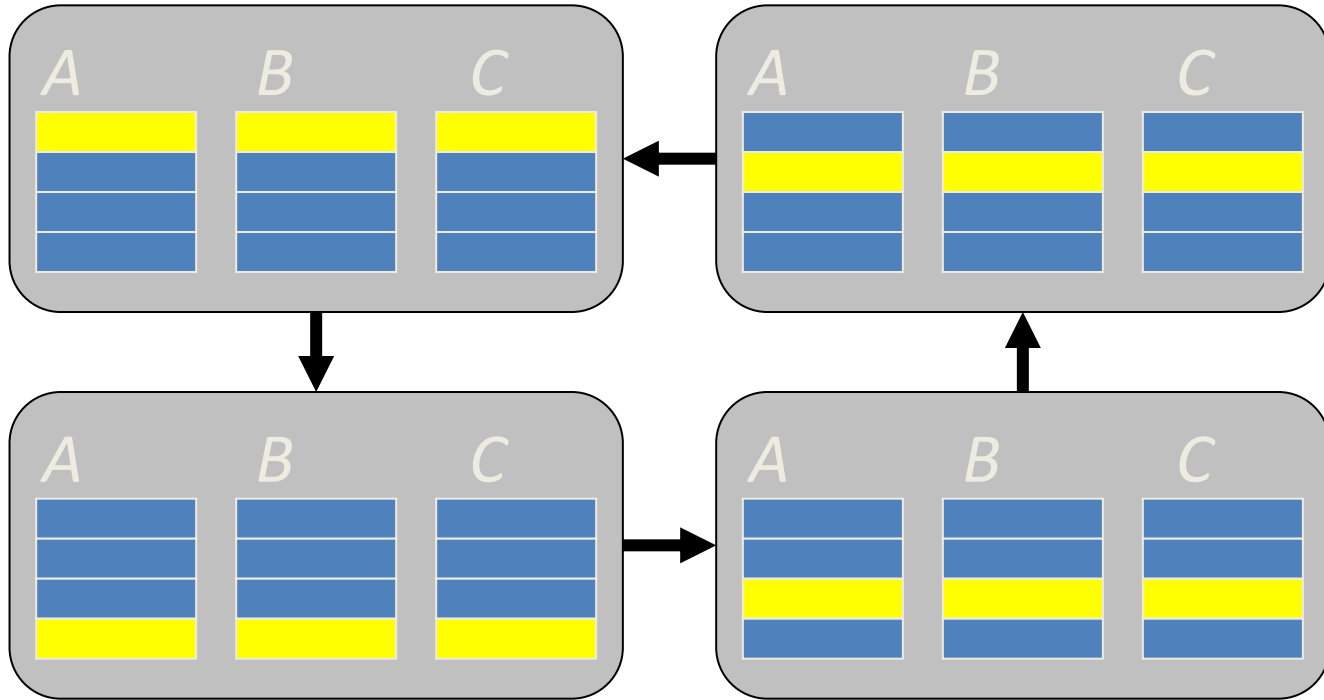
# Partitioning



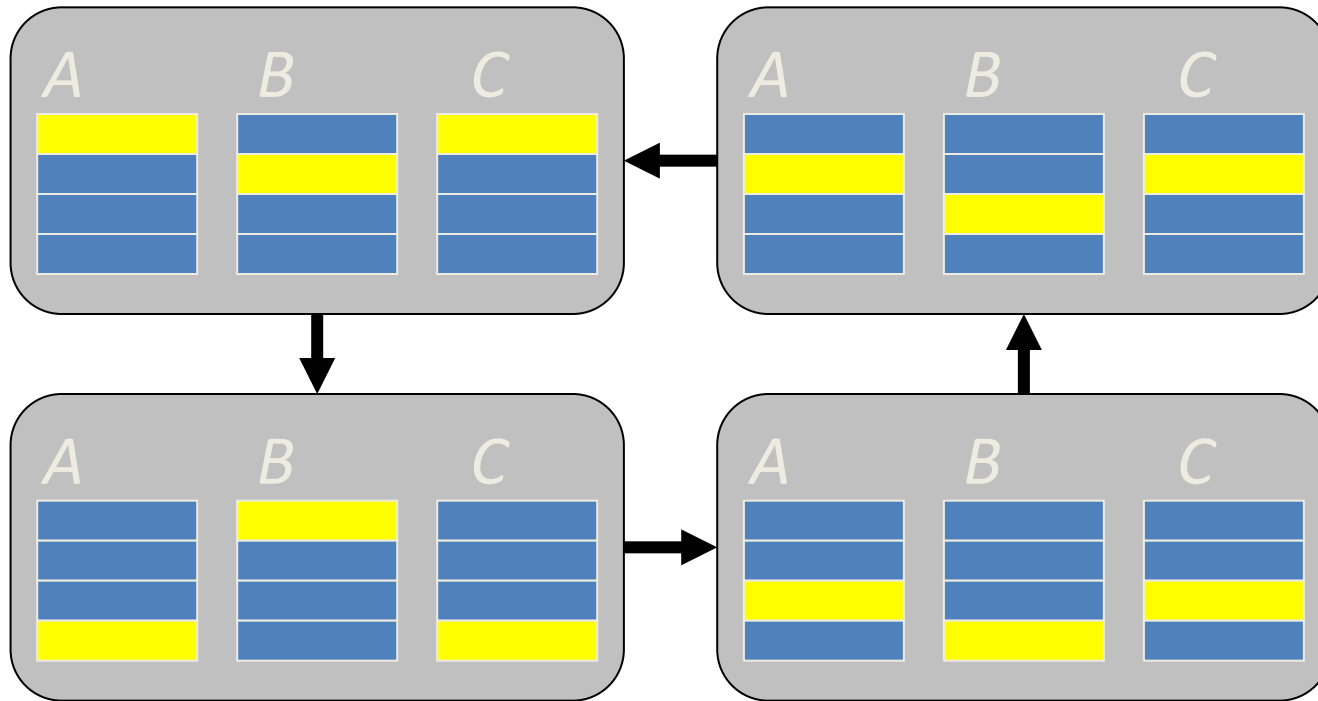
# Block-Striped Matrix Multiplication

- Partitioning
  - Divide matrices into  $p$  horizontal stripes of size  $(n/p) \times n$ .
  - Process  $i$  holds the  $i$ -th stripe of  $A$ ,  $B$ , and  $C$ , denoted as  $A_i$ ,  $B_i$ , and  $C_i$ , resp.
  - Each process has corresponding rows of three matrices.
- Communication
  - Each process must see every row of  $B$ .
  - Since Process  $i$  only starts with the  $i$ -th stripe of  $B$ , it must collect the other stripes through a sequence of shifts.
  - Organize processes into a *ring* to send and receive  $B$ 's rows.

# Communication of $B$

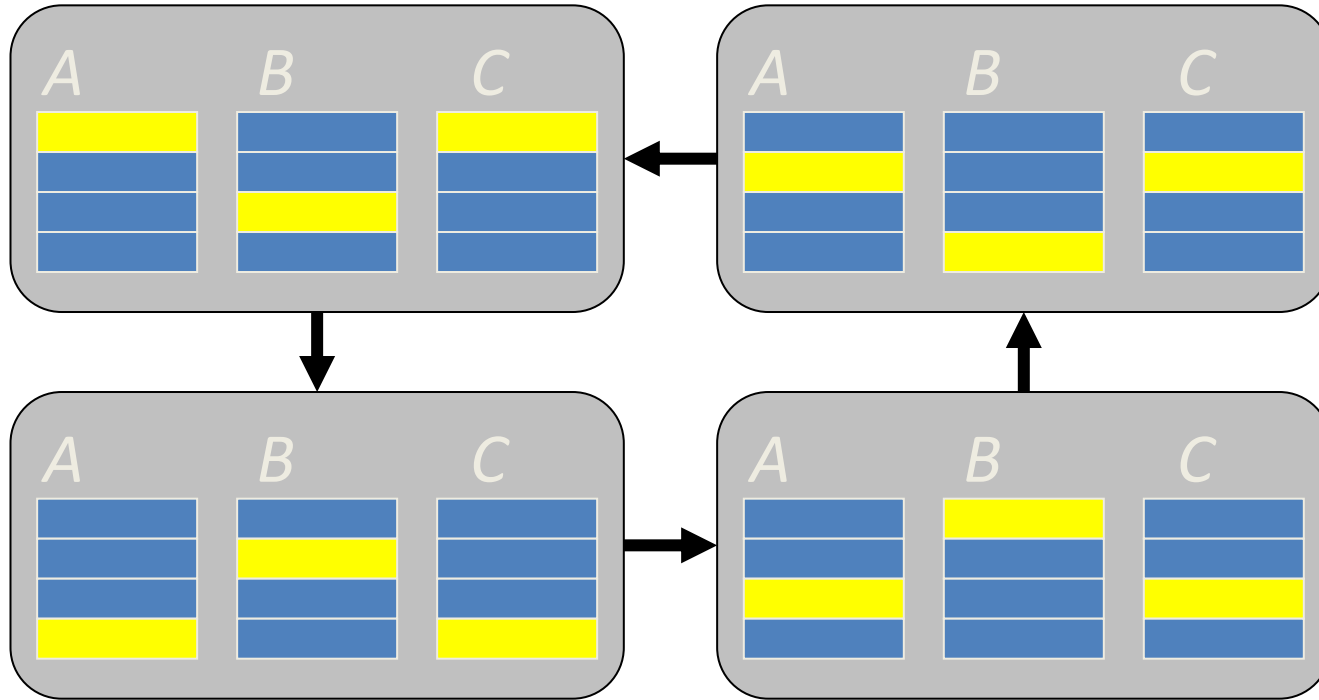


# Communication of $B$

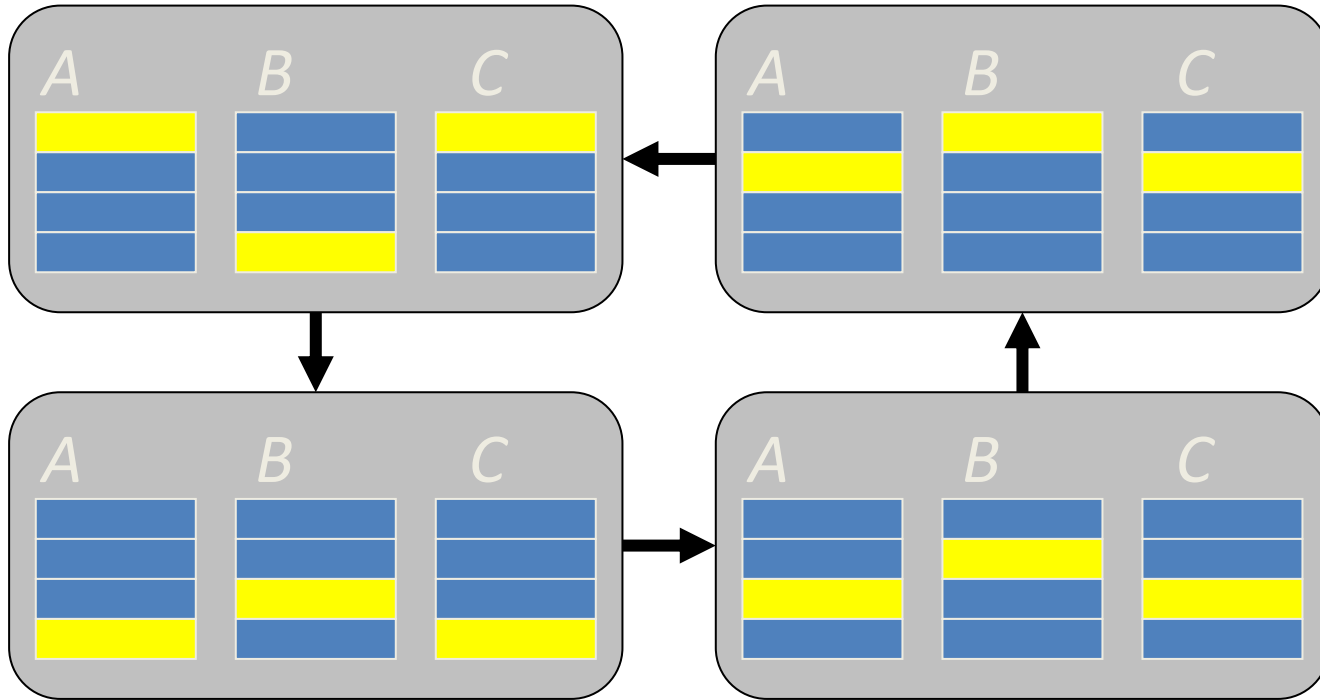




# Communication of $B$



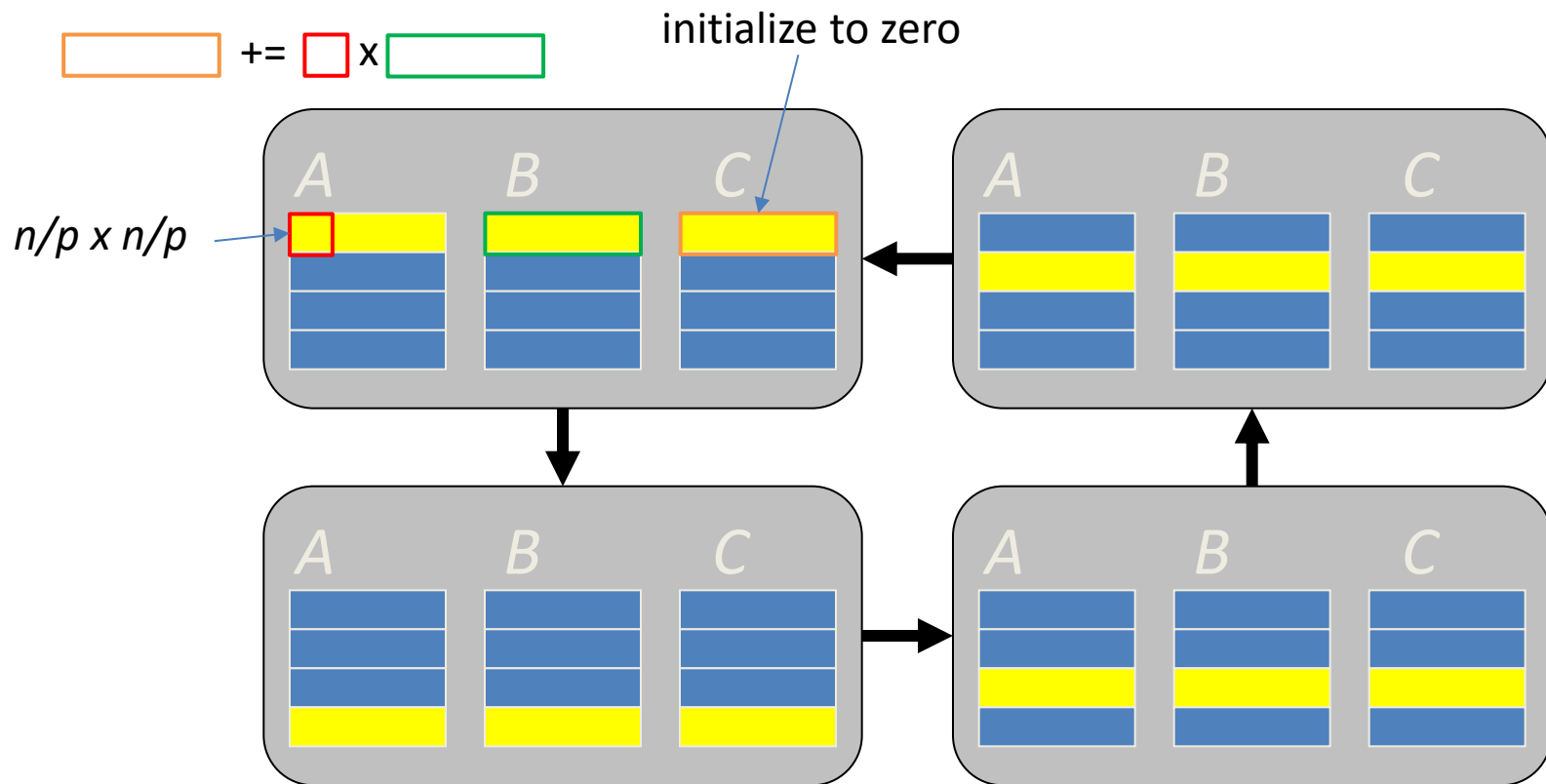
# Communication of $B$



# Computing $C$

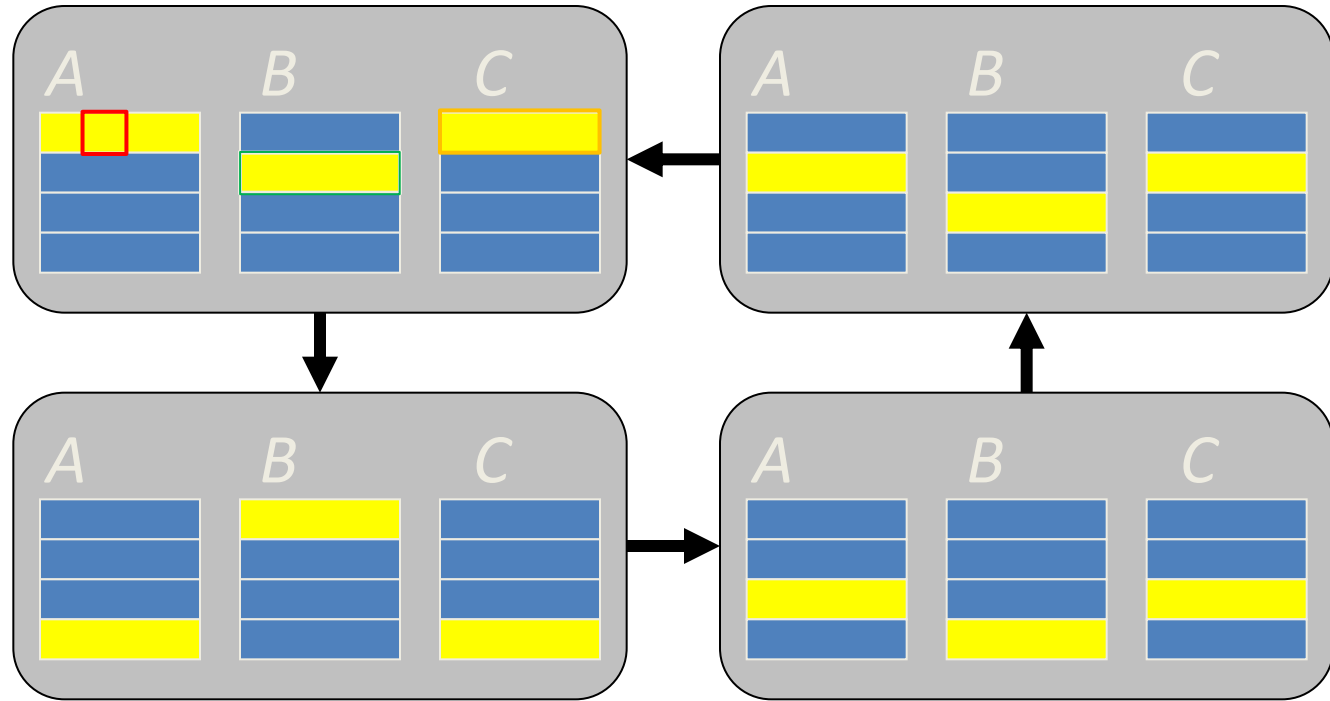
- Process  $i$  computes  $C_i = A_i^* \times B_i$ , where  $A_i^*$  refers to the specific columns of  $A_i$  that match the rows of  $B$  currently held by the processor.
- Process  $i$  sends its current block of  $B$  to its neighbor Process  $i+1$  and receives a new block of  $B$  from Process  $i-1$  (with Process  $p$  sending to Process 1 to form the ring).
- This process repeats  $p-1$  times.
- When every process has seen every block of  $B$ , and the local stripe  $C_i$  is complete.

# Computation of $C$



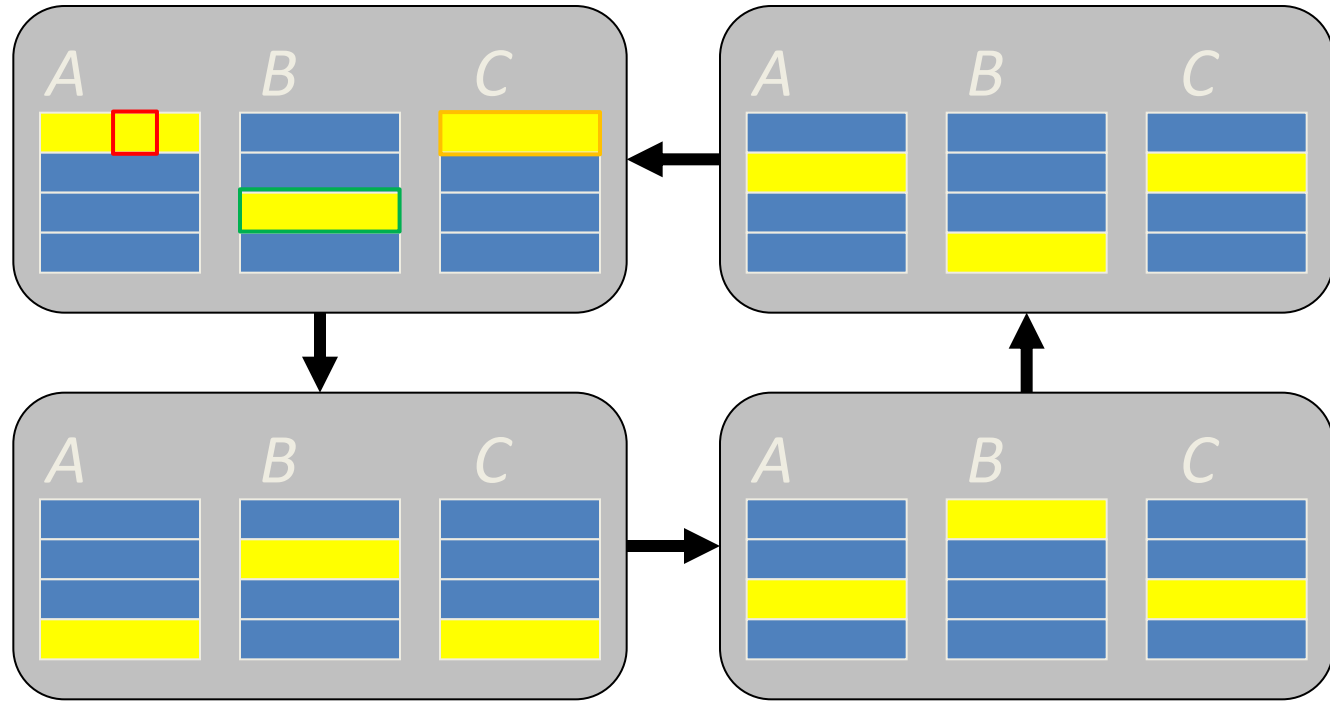
# Computation of $C$

$$\boxed{\phantom{000}} += \boxed{\phantom{00}} \times \boxed{\phantom{000}}$$



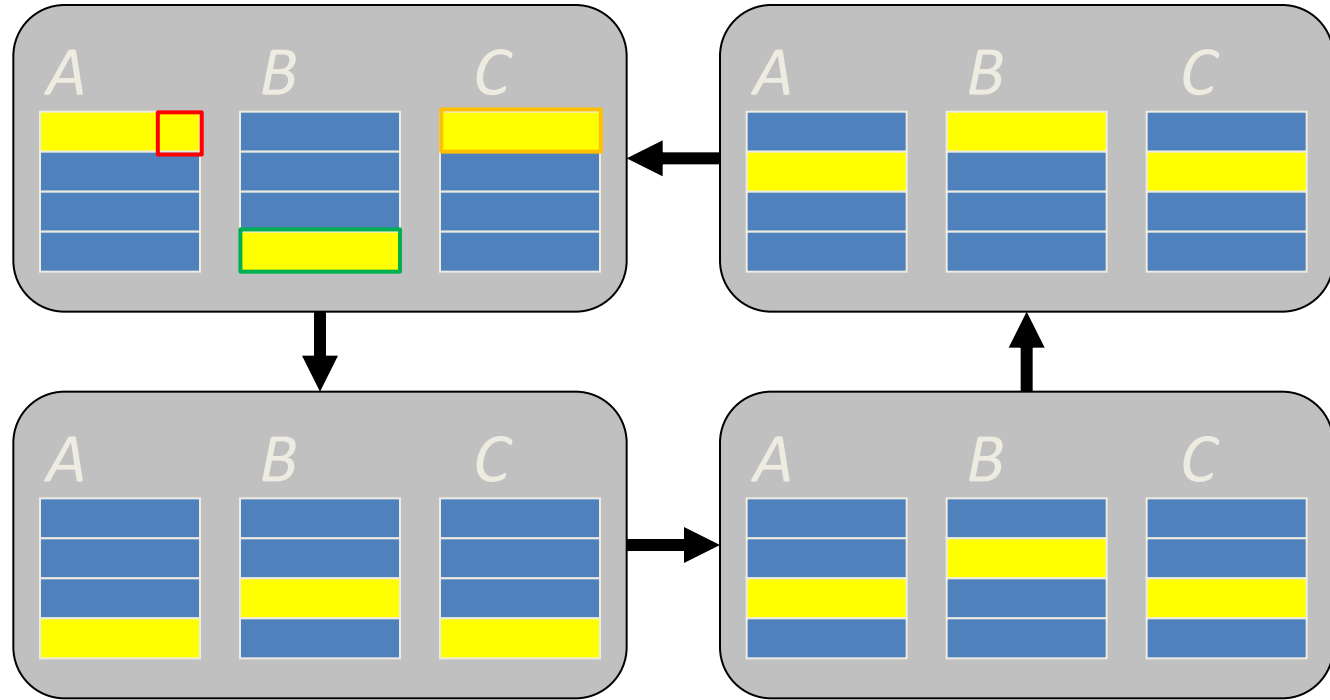
# Computation of $C$

$$\boxed{\phantom{000}} += \boxed{\phantom{00}} \times \boxed{\phantom{000}}$$



# Computation of $C$

$$\boxed{\phantom{000}} \mathrel{+=} \boxed{\phantom{00}} \times \boxed{\phantom{000}}$$



# Complexity Analysis

- Algorithm has  $p$  iterations
- During each iteration, each process multiplies

$(n / p) \times (n / p)$  block of  $A$  by  $(n / p) \times n$  block of  $B$ :

$$\Rightarrow O(n^3 / p^2) \quad \boxed{\phantom{000}} \text{ += } \boxed{\phantom{00}} \times \boxed{\phantom{0000}}$$

- Total computation time:  $O(n^3 / p)$
- Each process ends up passing  $(p-1) n^2 / p = O(n^2)$  elements of  $B$
- Ratio of computations per communication is poor:

$$(n^3 / p) / n^2 = n / p$$



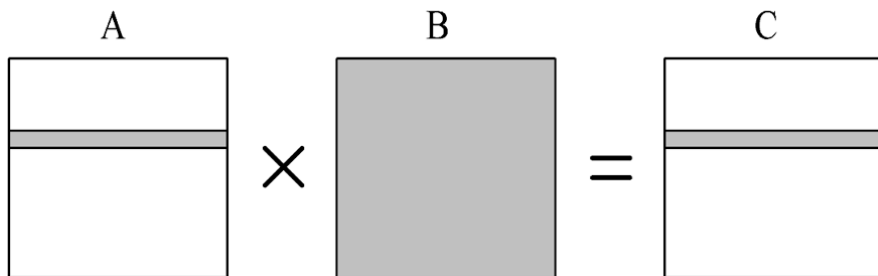


# Cannon's Algorithm

- Associate a primitive process responsible for a square (or nearly square) block of  $C$ .
- Uses a  $\sqrt{p} \times \sqrt{p}$  checkerboard decomposition, which is generally more scalable for large numbers of processors.
- Process:
  - Initial alignment (or *skewing*) phase followed by
  - Shift-multiply phases
- Computation-to-communication ratio rises to  $n/\sqrt{p}$ .

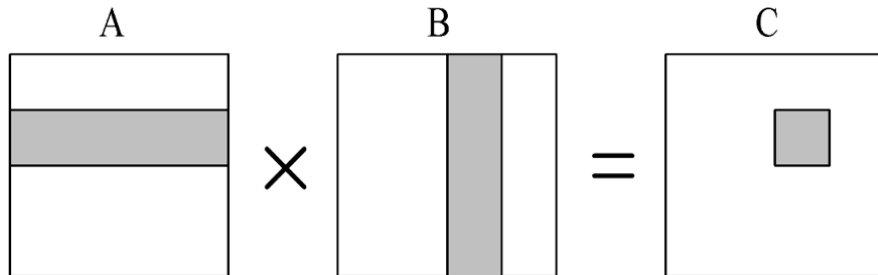
# Computing C

## Block-Striped Matrix Multiplication



Each process is responsible for computing  $n/p$  rows of  $C$ . It needs to reference  $n/p$  rows of  $A$  and every element of  $B$ .

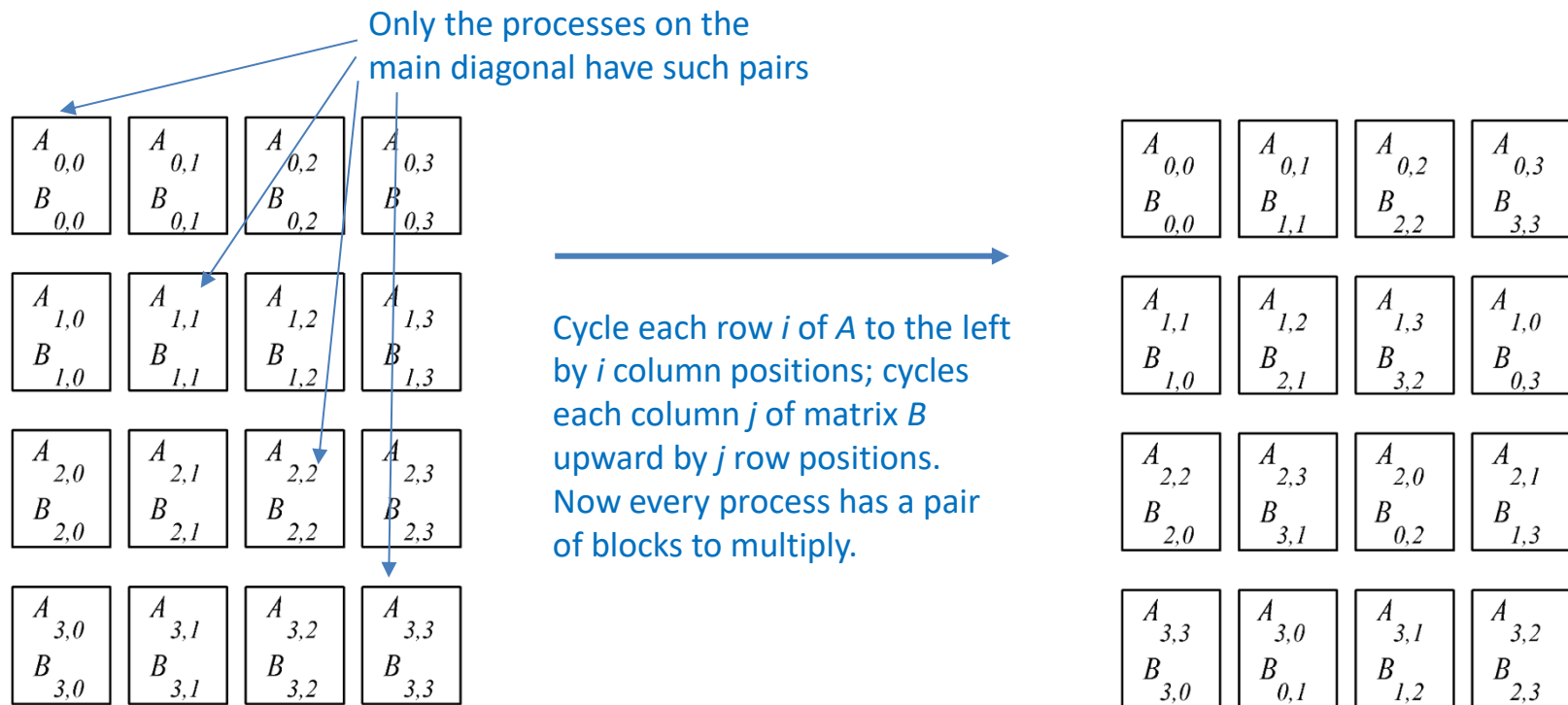
## Cannon's Algorithm



Each process is responsible for computing an  $(n/\sqrt{p}) \times (n/\sqrt{p})$  block of  $C$ . It needs to reference  $n/\sqrt{p}$  rows of  $A$  and  $n/\sqrt{p}$  columns of  $B$ .

# Need for Alignment

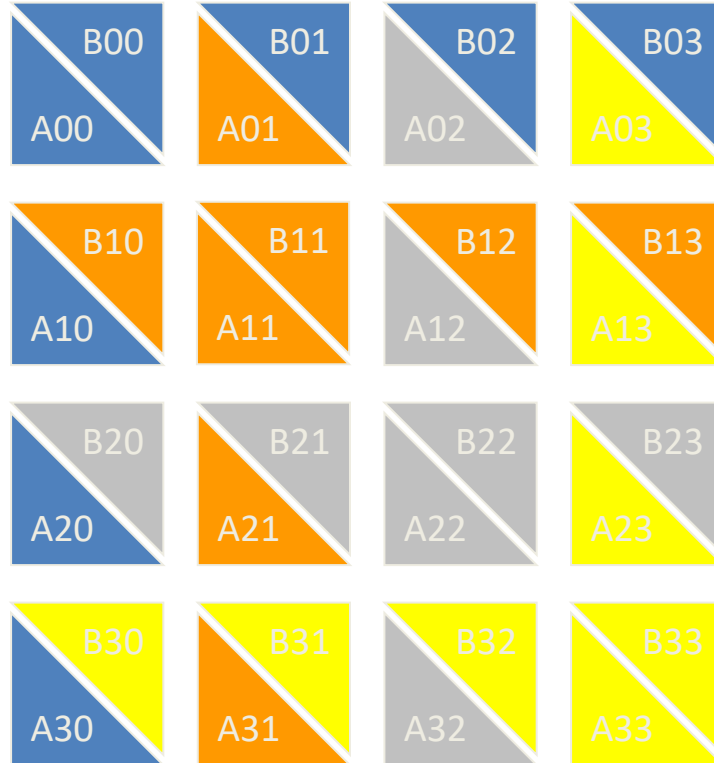
Recall block matrix multiplication:  $C_{ij} = \sum_{k=1}^m A_{ik} B_{kj}, \quad 1 \leq i, j \leq m$



# Blocks Need to Be Aligned

Each triangle  
represents a  
matrix block

Only same-color  
triangles should  
be multiplied



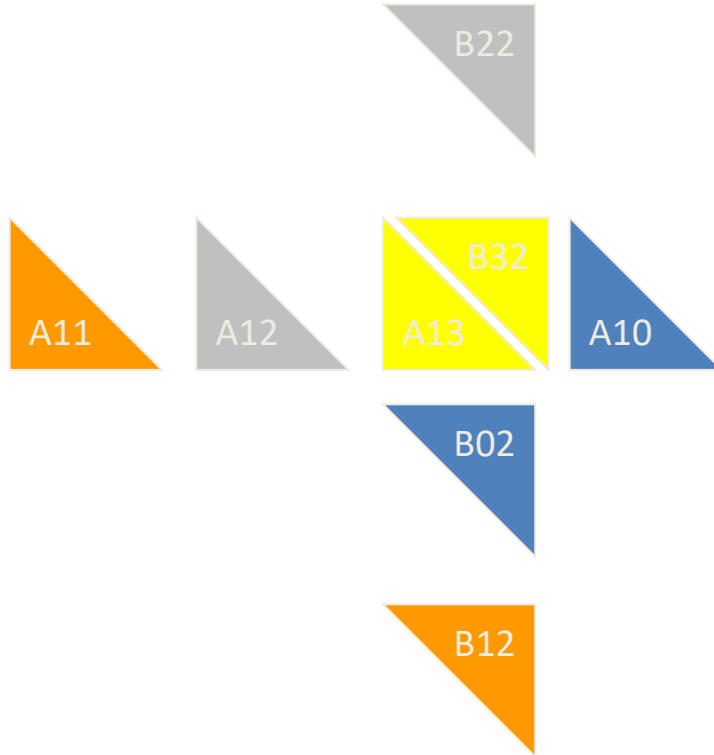
# Rearrange Blocks



Block  $A_{ij}$  cycles  
left  $i$  positions

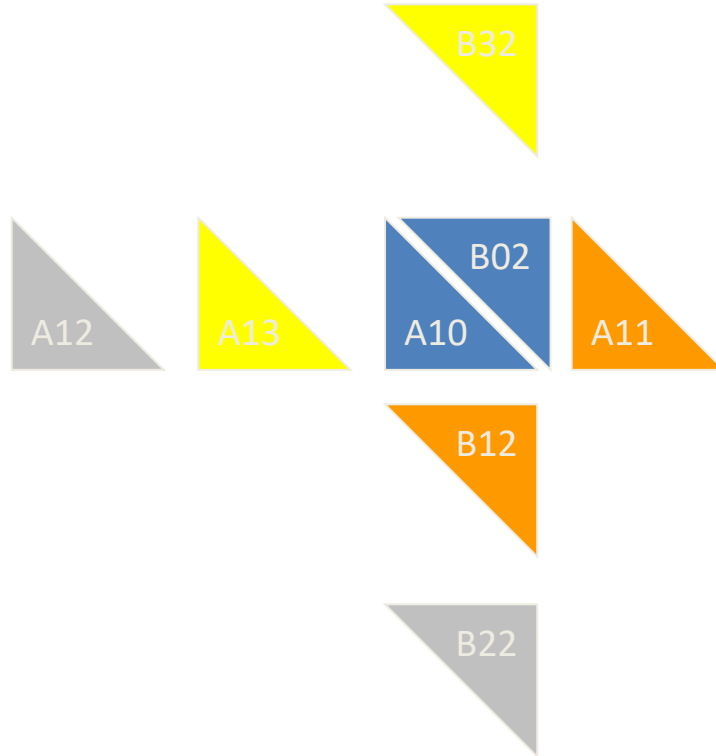
Block  $B_{ij}$  cycles  
up  $j$  positions

# Consider Process $P_{I,2}$



Step I

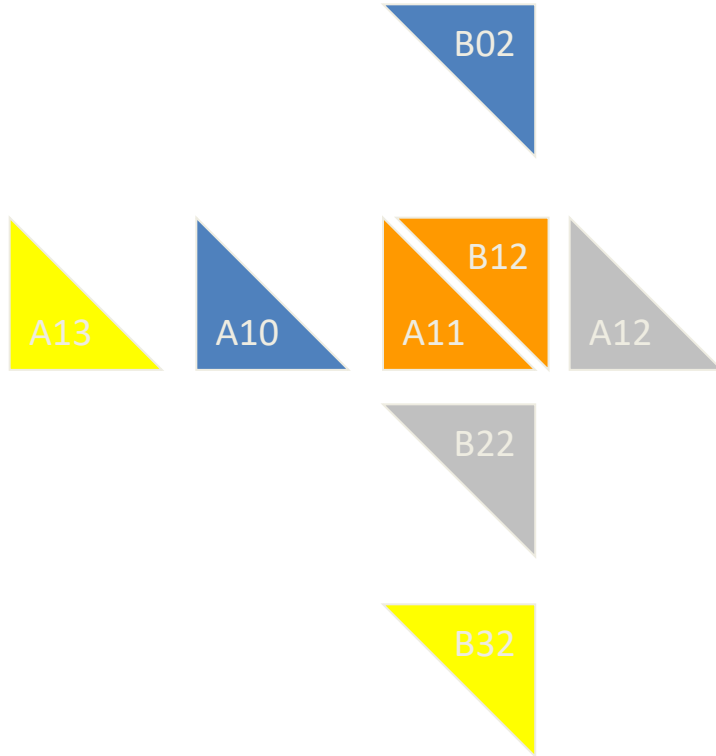
# Consider Process $P_{l,2}$



Step 2

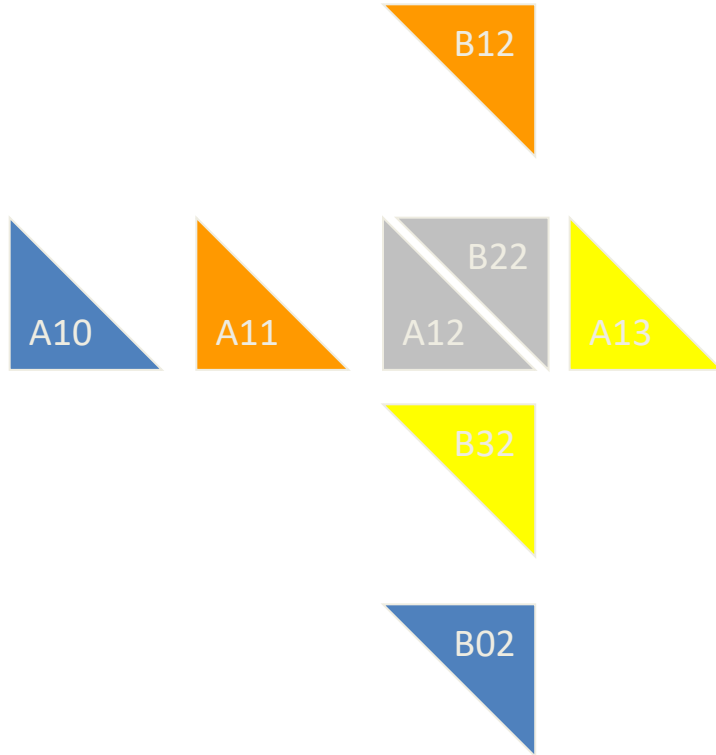


# Consider Process $P_{l,2}$



Step 3

# Consider Process $P_{l,2}$



Step 4

# Complexity Analysis

- Algorithm has  $\sqrt{p}$  iterations
- During each iteration, each process multiplies two  $(n / \sqrt{p}) \times (n / \sqrt{p})$  matrices:  $O(n^3 / p^{3/2})$
- Computational complexity:  $O(n^3 / p)$
- During each iteration, each process sends and receives two blocks of size  $(n / \sqrt{p}) \times (n / \sqrt{p})$
- Communication complexity:  $O(\sqrt{p} n^2 / p)$
- Ratio of computations per communication:  
$$(n^3 / p) / (\sqrt{p} n^2 / p) = n / \sqrt{p}$$



# Fox's Algorithm

- Uses a  $\sqrt{p} \times \sqrt{p}$  checkerboard decomposition.
- While Cannon's algorithm relies on shifting both matrices  $A$  and  $B$ , Fox's algorithm uses a broadcast-and-shift approach.
- It is often considered more intuitive than Cannon's because it mimics the outer-product definition of matrix multiplication more closely.

# Fox's Algorithm

- To compute a block  $C_{ij}$ , you need blocks from the  $i$ -th row of  $A$  and the  $j$ -th column of  $B$ .
- Fox's algorithm achieves this through two primary communication patterns in each step:
  - In each step  $k$ , one processor in each row broadcasts its block of  $A$  to all other processors in that same row.
  - After the multiplication, blocks of matrix  $B$  are shifted upwards by one position circularly.

# Step-by-Step Execution

- Each process  $P_{ij}$  starts with its assigned blocks  $A_{ij}$  and  $B_{ij}$ .
- Iteration  $k$  from 0 to  $q-1$  (where  $q = \sqrt{p}$ ):
  - In each row  $i$ , the process at column  $j = (i + k) \pmod{q}$  is designated as the broadcaster.
  - The broadcaster sends its local block of  $A$  to all other  $q-1$  processes in its row. Let's call this received block  $A_i^*$
  - Each processor  $P_{ij}$  multiplies the received  $A_i^*$  with its currently held block of  $B$ :
$$C_{ij} = C_{ij} + (A_i^* \times B_{\text{local}})$$
  - Every processor sends its current  $B$  block to the neighbor directly above it ( $P_{i-1,j}$ ) and receives a new  $B$  block from the neighbor below ( $P_{i+1,j}$ )

# Fox vs Cannon

- Don't have to deal with the complex initial skewing logic required by Cannon.
- Depending on the hardware, broadcast might be highly optimized by the network hardware, e.g., `MPI_Bcast` , making Fox's algorithm faster than the manual shifts of Cannon.
- But, requires an extra buffer to store the broadcasted A block, whereas Cannon is strictly in-place regarding communication buffers.



# Scalable Universal Matrix Multiplication Algorithm

Rewrite the matrix product into a sum of outer products:

$$C = \sum_{k=0} A_{*,k} \times B_{k,*}$$

where  $A_{*,k}$  is the  $k$ -th column of  $A$  and  $B_{k,*}$  is the  $k$ -th row of  $B$ .

Use *panels* of width  $b$ . Each step of the algorithm computes a *rank- $b$*  update:

$$C = C + A_{\text{panel}} \times B_{\text{panel}}$$

# SUMMA Setup

- Processes: Uses a  $\sqrt{p} \times \sqrt{p}$  checkerboard decomposition.
- Matrix: Process  $(i, j)$  owns blocks  $A_{ij}$ ,  $B_{ij}$ , and  $C_{ij}$ .
- Block size  $b$  chosen to balance communication latency and computational throughput (typically 32, 64, or 128).

# SUMMA Steps

for  $k = 0$  to  $n-1$  step  $b$ :

1. Determine which column of proc owns the current A-panel and which row of pro owns the current B-panel.
2. Broadcast the A-panel along rows
3. Broadcast the B-panel along columns
4. Rank-b update: Every processor multiplies the received panels

# Pipelined Communication

- Because the broadcasts happen on independent row and column communicators.
- Each process can start its multiplication while the next broadcast is already being prepared or sent by other nodes.
- Hides the communication latency.

